

線上實作課 · 第二堂

做一個自己的 AI Agent

不會寫程式，也能讓它真的上線

8/13 (四) 19:00-20:00 · 蔡子揚 Young



掃我 → 按 Fork

github.com/young-ai-courses/ai-agent-workshop

上週你做出來的東西



重點不是「會寫程式」，是你講兩句中文、它自己去做

今天在同一顆 repo 上往前接：讓它有畫面、有 AI、而且活在網路上

今天你帶走兩個能力

① Vibe coding

- 你說要什麼，AI 寫 code
- 你不讀每一行，你看結果對不對
- Codex 桌面版直接改你的檔案

② 搞定 server

- 部署上線（Vercel）
- 接 AI API（Groq）
- 存資料（Supabase）

AI 寫 code 這件事，走過三個階段

	2021 自動補全	2023 對話生成	2025 交辦執行
你做什麼	打字	問「怎麼寫」	說「我要什麼」
AI 做什麼	猜你下一行	給你一段，你貼回去	自己讀專案、改檔、開 PR
你的角色	打字的人	組裝的人	需求方 + 驗收方

Codex · Claude Code · Cursor 都在第三階段

所以瓶頸換位置了

以前的瓶頸

- 你會不會寫
- 看得懂語法嗎
- 記得住 API 嗎

現在的瓶頸

- 你到底要什麼
- 看不看得出它做錯
- 說不說得清楚

上週你已經寫過規格了

上週你做的

- 跟 ChatGPT 聊出需求
- 貼成 GitHub Issue
- Codex 讀它、寫 code

今天沿用

- 同一份規格，不重寫
- 換成一個要上線的東西
- 重點在部署、存資料、調 AI

規格怎麼寫、怎麼 review —— 上週那套照用，今天不重講

「需求講不清」比 AI 早了五十年

年代	解法	為什麼還是會出事
1970s 瀑布式	先寫超厚規格書，簽核才 開工	寫的時候還不知道要什麼
2000s 敏捷	不寫厚規格，兩週一版邊 做邊修	靠「人的常識相通」補洞
2025 SDD	規格是唯一真相，code 是它的產物	← 我們現在在這

敏捷能省規格，是因為人有常識

你跟工程師說「做登入頁」

- 他自己就知道要有密碼欄
- 要遮起來、打錯要提示
- 不能把密碼印進 log

你跟 AI 說「做登入頁」

- 它給你統計上最常見的做法
- +一堆你沒要的東西
- +漏掉你以為理所當然的

那些沒人寫在規格上的事，工程師自己補了；AI 不會——它只會猜

SDD 一句話

規格是你唯一真的要親手 review 的東西
code 是它的產物

code 壞了可以重新生成一次
規格錯了，生成幾次都是錯的

所以 PRD 怎麼寫？你不用學會寫



壞消息：你不知道要寫什麼（第一次寫 PRD 的人都不知道）

好消息：你不用知道 —— LLM 知道要問什麼



這支 prompt 是今天最重要的工具

開 ChatGPT 或 Claude 的網頁版，開一個新對話貼進去（不是 Codex）

你是資深產品經理，把外行人模糊的願望，
問成工程師做得出來的需求單。

我的願望：「_____」

硬限制：不會寫程式／只有 3.5 小時／
只有一個輸入框＋一次 AI 呼叫

規則：一次只問一題，最多 6 題

只問使用者層面，不問技術問題

我說「不知道」就給我 2-3 個選項

問完產出 PRD，要有「做得完 vs 做不完」

全文（含 PRD 完整格式）在 repo → docs/02-prompts.md 的 Prompt 1，複製就好

它給你 PRD 之後：逐條 review

A 有沒有搞懂

是我要的東西嗎
有沒有自己加東西

B 夠不夠具體

「他做 X → 看到 Y」
沒有無法打勾的形容詞

C AI 做什麼

輸入、輸出格式
不確定的怎麼處理

D 做不完的呢

有沒有誠實切開
做不完的有寫下來嗎

E 有沒有踩線

個資・金流・責任
key 有沒有進 code

F 能不能打勾

三條驗收條件
當場打得勾或打得叉

最容易漏的是 D：它不會主動說「做不完」

- 你說要五個功能 → 它五個都寫進 PRD，都寫得很可行
- 因為它不知道你只有 3.5 小時、不會寫程式

所以你要主動問一句：

**「我只有 3.5 小時、不會寫程式、只有一個輸入框加一次 AI 呼叫。
重新切一次：今天做得完的一組，做不完的另外列清單」**

做不完的不要刪掉 —— demo 時你可以說「這版先做到 X，接下來長 Y」

接下來這條線，全自動



你只做兩個動作：把 SPEC 給 Codex、看過 diff 說 OK
push 之後每一步都是自動發生的——這叫 CI/CD

（正式團隊還會加一層 PR review，今天走最短路先讓它活著）

我現在從零示範一次

貼在同一個地方：ChatGPT 桌面版 → Codex → 對話框 → 送出

請幫我把這個專案部署上線。我不會用終端機，
每一步先講你要做什麼，再執行。

1. 進到 starter-kit 資料夾再裝套件、再部署
一定要在 starter-kit 裡部署，站在最外層會「成功」但打開是 404
2. 部署到 Vercel production；問到專案根目錄要選哪個，選 starter-kit，其餘用預設
3. 要加 GROQ_API_KEY 的時候停下來告訴我，我自己去 Vercel 貼
不要問我 key、不要把它寫進任何檔案或印出來
4. 我說加好了，你再部署一次 production 讓它生效
5. 把網址給我，並確認打開不是 404

key 你自己到後台貼，不經過 AI 對話 —— 這是今天唯一不能外包的事

網站 → Vercel 專案 Settings → Environment Variables | 排程 → GitHub repo Settings →
Secrets and variables → Actions

把 SPEC 交給 Codex 改

規格是你寫的，動手的是它 —— 你只負責看它做對沒有

- 1 在 Codex 貼上你 review 過的 SPEC，加一句「先複述你的理解，等我說開始」
- 2 看它複述得對不對 —— 不對就先講清楚，別讓它開工
- 3 它改完會給你 diff（哪幾個檔、改了什麼）→ 你看過再接受
- 4 接受後 push → Vercel 自動重新部署，30 秒後重新整理你的網址

⚠ Codex 額度依方案、模型、任務大小而變 —— 一次交辦一件小事最省
快用完時改用較小的模型，或看 usage dashboard；用光看 QUICKSTART 的 Plan B

你做的東西，長這樣

橘色是你的（改得動） · 灰色是租來的服務（只能呼叫）



①②③ 是你的，你改得動；④⑤ 是租來的，你只能照它的規則呼叫

想法落在哪一層，就叫 AI 動那一層

你不必知道要改哪個檔 —— 講你想看到什麼就好

你的想法	落在	你就這樣跟 Codex 講
畫面上的字、顏色、按鈕	① 頁面	「送出按鈕的字改成『幫我整理』」
AI 回答的內容或格式	③ 伺服器	「回答多一欄負責人，沒寫就填未指定」
要記住上次打了什麼	⑤ 資料庫	「我要記住上次打的，該加什麼？」

說得出「從什麼變成什麼」→ 直接講，這叫具體

不知道該動哪裡 → 先問路：「我要 __，該加什麼？」，這叫抽象

那個 AI 在你的網站哪裡？



為什麼 key 不能放前端：任何人打開瀏覽器的原始碼就看得得到它
所以前端只跟「你自己的伺服器」說話，由伺服器拿著 key 去問 AI

Chatbot 跟 Agent 差在呼叫幾次

今天做到哪一步，我講清楚，不含糊

	今天做的	完整的 Agent
API 呼叫	一次	多次，它自己決定要幾次
「自我檢查」	同一段輸出裡自己說「我不確定 X」	真的跑第二輪驗證第一輪
工具	沒有	自己呼叫工具查資料、讀檔
下一步	你決定	它看上一輪結果決定

今天裝的是骨架：角色 + 判斷規則 + 自我檢查

讓它自己跑多輪（五層裡的 ④ Loop）是下一步 —— 今天不做，但你要知道差在哪

Agent 零件①：角色設定

跟 Codex 說：把 AI 的角色設定換成這段 —

你是一位嚴謹的會議秘書。

把使用者貼進來的筆記整理成三類：

待辦事項 / 已做的決議 / 需要再確認的事。

用表格輸出，欄位是「類別 | 內容 | 負責人」。

沒寫負責人的填「未指定」，不要自己猜是誰。

不確定的放進「需要再確認」，不要編。

「不要自己猜」「不要編」這兩句很重要 —— 不寫它就會編

Agent 零件②③：判斷規則 + 自我檢查

跟 Codex 說：AI 的角色設定再加這幾條規則 —

- 沒寫負責人的填「未指定」，不要自己猜是誰
- 只根據我貼的內容，不要補你覺得應該有的事項
- 不確定屬於哪一類，就放進「待確認」
- 最後另外列一段「我不確定的地方」給我看

同一段筆記，它從「編出一堆合理但根本沒講過的待辦」
變成「老實承認這裡我不知道」—— 差別只在這四行

裝完之後：調參數讓它更穩

跟 Codex 說：

「把 temperature 設成 0.2，max_completion_tokens 設成 1200」

temperature 沒設的話預設是 1，調低 = 每次結果更穩、少亂發揮

max_completion_tokens 輸出上限，避免它寫成一篇論文

model 現在用 llama-3.3-70b-versatile，免費又快

- 輸出格式每次都跑掉 → 先叫它把 temperature 調到 0.2，不是換 model
- 答案被截斷 → 叫它把 max_completion_tokens 加大
- 真的需要更強的推理 → 才叫它換更大的 model（會變慢）

改東西的兩條紀律

① 一次只講一件事

- ✗ 「幫我做一個會議整理器」
- ✗ 「介面弄漂亮一點」
- ✓ 「把標題改成 X，提示改成 Y」

② 壞了先退回，不要接著補

- ✗ 「幫我修好」 → 它會一直加東西，越補越難救
- ✓ 「先找出上一個能動的版本，說明要退哪些檔，等我確認」
- 退回去之後，把需求重講一次

退回去不算失敗 —— 版本控制的整個意義，就是讓你敢改

你的網站現在沒有記憶

- 重新整理 → 剛才打的東西不見了
- 關掉分頁 → 全部歸零
- 因為資料只活在瀏覽器的記憶體裡，沒有人把它寫下來
- 要留住 = 要有一個地方存 = 資料庫

今天不做完 —— 下一頁給你要怎麼交辦，回家自己接

要存資料的時候：交辦這一段

我要讓網站記住使用者之前送出過的內容。

請幫我接 Supabase（走 Vercel Marketplace，環境變數會自動注入），
建一張表存紀錄，然後：

1. 建完表一定要開 RLS，並加上讀寫的 policy
2. 告訴我怎麼確認 RLS 真的生效

不要把 key 印在畫面上。

● anon key 本來就會出現在前端，它不是秘密 —— 真正的邊界是 RLS + 最小權限 policy
建表後先開 RLS；在你寫出明確 policy 之前，前端不該讀得到任何資料

你剛才做的事，有名字

我用來教學的五層地圖（不是官方標準名詞）—— 你今天走過前三層

- | | | | |
|---|---------------------|--|----------------------------|
| 1 | Prompt Engineering | | 你寫的那些 prompt |
| 2 | Context Engineering | | 你 review 過的 SPEC.md 就是一個例子 |
| 3 | Harness Engineering | | CI、權限、工具限制都算，綠勾勾是一種 |
| 4 | Loop Engineering | | 讓系統自己一輪一輪推進（還沒做） |
| 5 | Graph Engineering | | 多個 AI 分工協作（還沒做） |

123 今天你親手做過了

上台 2 分鐘，講這三件事

- 1 我原本要花多久做這件事
- 2 這個東西幫我做掉了哪一段
- 3 今天沒做完的部分，接下來會怎麼長

github.com/young-ai-courses/ai-agent-workshop

QUICKSTART 六張工單：環境 → 上線 → 調 AI → 存資料

agent-prompts Agent prompt 模板

AGENTS.md clone 完 Codex 就是你的助教